Terraform Fundamentals

Instructor: Dyego Souza
Findmore Academy

Trainning Overview

- 5 Sessions (2h each)
- 10 hours total
- Format: Live Online (Moodle)
- Level: Beginner
- Target Audience: Developers, Cloud Engineers, DevOps, SysAdmins
- Theory + Hands-on Labs
- Tools: GitHub, Azure

Learning objectives

- Session 1: Infrastructure as Code Fundamentals and Introduction to Terraform
- Session 2: Working with Variables and Resources
- Session 3: Advanced Resources and Providers
- Session 4: Modules, Loops, and Code Reusability
- Session 5: Integration with GitHub Actions and CI/CD Pipelines

Interno

Introduce yourself

Dyego Souza

- DevOps & Cloud Engineer
- Consultant
- Azure 5x and Terraform Associate Certified
- https://www.linkedin.com/in/dyegodesouza/
- ramehx@yahoo.com.br

Interno

SESSION 1: INFRASTRUCTURE AS CODE FUNDAMENTALS AND INTRODUCTION TO TERRAFORM

Interno

What is Infrastructure as Code (IaC)?

- Infrastructure as Code (IaC) is the practice of defining and managing infrastructure through source code, which can be treated and maintained like any other software system.
- Infrastructure includes compute (e.g., virtual machines), networking, security components, and managed cloud services such as Kubernetes clusters, serverless platforms, and even non-traditional resources like GitHub repositories.
- IaC code must be stored in source control systems to enable versioning, auditability, collaboration, and full integration with CI/CD pipelines.
- While IaC is a natural fit for cloud environments, it can also be applied to on-premises and virtualized infrastructures.

```
# Resource Azure virtual machine
resource "azurerm_linux_virtual_machine" "mylinuxvm" {
    name = "mylinuxvm-1"
    computer_name = "devlinux-vm1" # Hostname of the VM
    resource_group_name = azurerm_resource_group.myrg.name
    location = azurerm_resource_group.myrg.location
    size = "Standard_DS1_v2"
    admin_username = "azureuser"
     network_interface_ids = [
        azurerm_network_interface.myvnic.id
        ]
  admin_ssh_key {
    username = "azureuser"
    public_key = file("${path.module}/ssh-keys/terraform-azure.pub")
  }
  os_disk {
    name = "osdisk"
    caching = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
source_image_reference {
  publisher = "RedHat"
    offer     = "RHEL"
    sku       = "83-gen2"
    version   = "latest"
}
custom_data = filebase64("${path.module}/app-scripts/app1-cloud-init.txt") # Custom
}
```

Interno

Benefits of Infrastructure as Code

Automation and Consistency

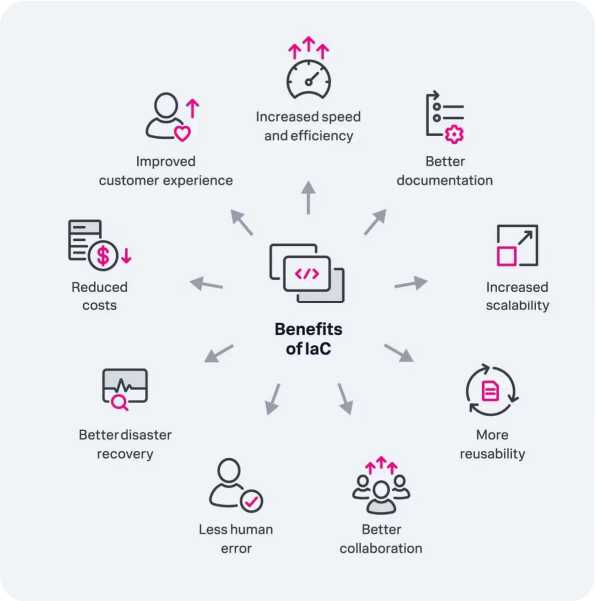
IaC automates infrastructure management, ensuring consistent and repeatable deployments across environments.

Version Control Benefits

Using code for infrastructure allows for version control, tracking changes and enabling collaboration.

Scalability and Reduced Errors

IaC supports scaling infrastructure efficiently while minimizing manual errors in deployment processes.



Fonte: https://www.splunk.com/en_us/blog/learn/infrastructure-as-code-iac.html

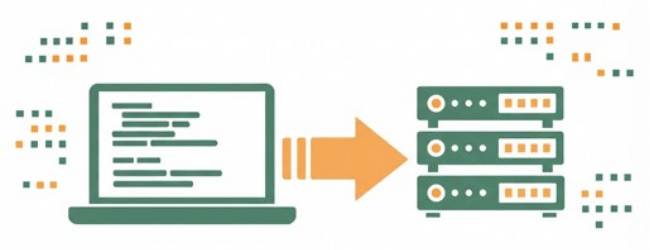
Interno

How Infrastructure as Code Works?

Infrastructure as Code (IaC) automates infrastructure configuration, reducing inconsistencies and delays in deployments by defining infrastructure through code.

Typical IaC workflow:

1. The team writes infrastructure configurations using a declarative or programming language (such as Terraform HCL).
2. The configuration files are stored and versioned in a source code repository.
3. An IaC tool executes the code and provisions, updates, or destroys infrastructure as required.



Interno

Imperative x Declarative Configuration

Aspect	Declarative Configuration	Imperative Configuration
Core idea	Describe **what** the desired end state should be	Define **how** to achieve the desired state step by step
Focus	Desired state	Execution process
Idempotency	Naturally idempotent: applying the same configuration multiple times produces the same result	Not inherently idempotent unless explicitly coded
State management	Tool compares current state vs desired state	State is managed manually or implicitly
Examples	Terraform (HCL), CloudFormation, ARM/Bicep, Kubernetes YAML	Linux Bash scripts, PowerShell, Python scripts, Network equipment CLI
Typical use cases	Infrastructure provisioning, cloud resources, environments	System configuration, orchestration, procedural automation
Error handling	Tool handles drift and reconciliation	Requires manual checks and condition handling

Interno

Main IaC Tools

Terraform

Terraform is a popular tool for infrastructure provisioning using declarative configuration language.

AWS CloudFormation Use

AWS CloudFormation enables infrastructure as code for automating AWS resource management.

Ansible Capabilities

Ansible automates software provisioning, configuration management, and application deployment.

Chef Configuration

Chef manages infrastructure through code and supports complex automation workflows.

Interno

Proprietary vs Provider-Agnostic

Proprietary IaC:

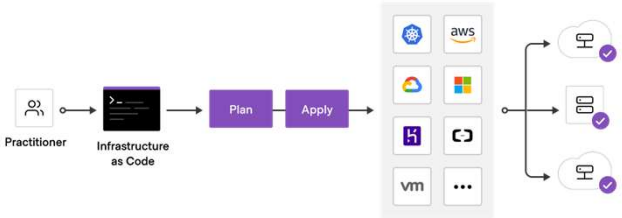
- Tightly coupled to a single cloud provider or platform
- Uses vendor-specific languages and services
- Limits portability and increases vendor lock-in
- Examples: AWS CloudFormation, Azure ARM / Bicep

Provider-Agnostic IaC

- Designed to work across multiple cloud providers and platforms
- Uses a unified language and workflow
- Abstracts provider-specific APIs behind a common interface
- Examples: Terraform, Pulumi

Advantages of Provider-Agnostic IaC

- Reduced vendor lock-in
- Consistent tooling and workflows across clouds
- Easier multi-cloud and hybrid-cloud strategies
- Reusable modules and shared best practices



Interno

What is Terraform?

Terraform is an open-source **Infrastructure as Code (IaC)** tool created by HashiCorp.
It enables infrastructure provisioning and management using a **declarative approach**.

HashiCorp Terraform

https://registry.terraform.io/

Key Characteristics

- **Multi-provider & Multi-platform**
  Works across AWS, Azure, GCP, Kubernetes, SaaS platforms, and more.
- **Extensive Provider Ecosystem**
  Large ecosystem of official and community-maintained providers.
- **Declarative Language (HCL)**
  Uses HashiCorp Configuration Language (HCL), designed to be human-readable and easy to maintain.
- **Reusable and Maintainable Code**
  Supports variables, modules, and composition patterns.
- **Terraform Core**
  Responsible for parsing configuration, building execution plans, and managing dependencies.
- **State Management**
  Maintains infrastructure state to track real-world resources and detect drift.
- **CLI-Based Workflow**
  Command-line interface for initialization, planning, applying, and destroying infrastructure.
- **Extensible Architecture**
  Custom providers can be developed using **Go**, extending Terraform to new platforms and APIs.

Interno

HCL – HashiCorp Configuration Language

- HCL is a **declarative configuration** language created by HashiCorp.
- Designed to be human-readable, concise, and easy to maintain.
- Used by Terraform to describe the **desired state of infrastructure**.
- Combines a simple syntax with structured data types.

Supports:

- Blocks (e.g., resource, provider, module)
- Expressions and functions
- Variables, outputs, and locals
- Balances **readability** and **machine parsing**, unlike JSON-only configurations.

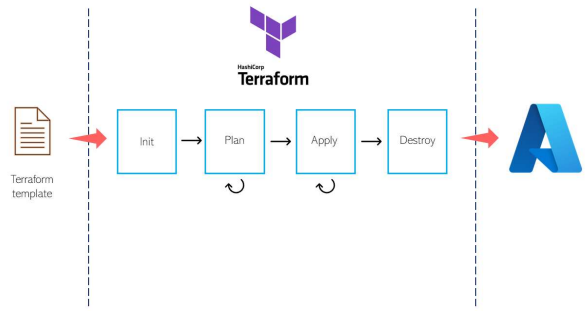
main.tf

main.tf > resource "azurerm_resource_group" "rg" > name

```
# Provider declaration
provider "azurerm" {
  features {}
}

# Input variables
variable "location" {
  description = "Azure region to deploy resources"
  type        = string
  default     = "East US"
}

variable "storage_accounts" {
  description = "List of Storage Account names"
  type        = list(string)
  default     = ["stterraform01", "stterraform02"]
}

# Resource Group
resource "azurerm_resource_group" "rg" {
  name     = "RG-1"
  location = var.location
}

# Storage Accounts using loop (for_each)
resource "azurerm_storage_account" "storage" {
  for_each = toset(var.storage_accounts)

  name                     = each.value
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"

  tags = {
    environment = "demo"
  }
}
```

Interno

Getting Started: Installation and First Commands



Workflow Steps

1. **Write** configuration files **(.tf)**
2. **terraform init:** Initialize project
3. **Terraform fmt:** Apply code formating and indentation
4. **Terraform validate:** Perform local basic code validation
5. **terraform plan:** Preview changes
6. **terraform apply:** Apply changes
7. **terraform destroy:** Destroy resources

Fonte: https://okuzayoxm7schematic.z21.web.core.windows.net/terraform-generate-architecture-diagram.html

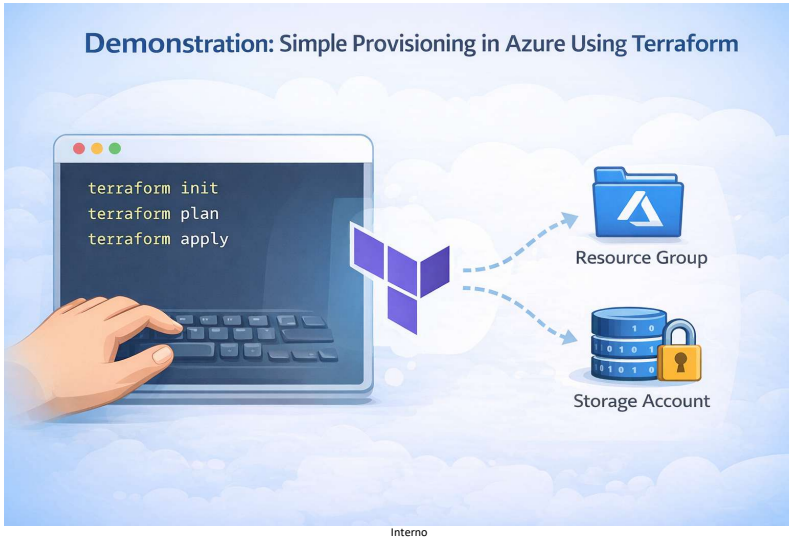
Installing Terraform is the first step to managing infrastructure as code with a simple setup process.

https://developer.hashicorp.com/terraform/install

FINDMORE ACADEMY

Interno

Lab 1: Simple Provisioning in Azure Using Terraform



Interno

Terraform Init

```
@root ~ % terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.27"...
- Installing hashicorp/aws v3.74.3...
- Installed hashicorp/aws v3.74.3 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary
```

Fonte: https://sidtechtalks.in/terraform-script-for-aws-with-examples/

› Terraform Init

- 'init' command sets up the working directory and downloads necessary plugins for Terraform.
- Run init again everytime you update or add a new provider, module or version
- OBS: The terraform.lock.hcl file, also known as the dependency lock file, is automatically created by Terraform to record the exact versions of providers used in your configuration.
- This ensures consistency and prevents issues that may arise from using different versions across different environments or team members.

Interno

Terraform Format (fmt)

```
resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
}

resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_subnet" "subnet" {
  name                 = var.subnet_name
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]


}
```

> Terraform fmt

- 'fmt' command applies code formatting and indentation for better readability and consistency.
- It is an optional step, but recommended for improving code quality and it fixes some errors that would raise on the plan step
- terraform fmt –recursive : applies formatting also to subfolders

FINDMORE ACADEMY

Interno

Terraform Validate

> Terraform validate

- Validate configuration syntax and internal consistence.
- OBS: does not check configuration errors and conflicts against infrastructure state

```
sumeetninawe@Sumeets-MacBook-Pro tfvalidate % terraform validate

Error: Reference to undeclared input variable

  on main.tf line 7, in resource "aws_instance" "my_vm":
   7:     Name = var.tag_name_mistake,

An input variable with the name "tag_name_mistake" has not been declared. This variable can be declared with a
variable "tag_name_mistake" {} block.
```

Fonte: https://spacelift.io/blog/terraform-validate

FINDMORE ACADEMY

Interno

Terraform Plan

```
# module.secret-manager.aws_secretsmanager_secret_version.[redacted]["json"] will be destroyed
- resource "aws_secretsmanager_secret_version" "sm-[redacted]" {
    - arn            = "arn:aws:secretsmanager:us-east-1:[redacted]:secret:[redacted]" -> null
    - id             = "[redacted]" -> null
    - secret_id      = "[redacted]" -> null
    - secret_string  = (sensitive value)
    - version_id     = "[redacted]" -> null
    - version_stages = [
        - "AWSCURRENT",
      ] -> null
  }
# module.secret-manager.aws_secretsmanager_secret_version.[redacted] | will be created
+ resource "aws_secretsmanager_secret_version" "sm-[redacted]" {
    + arn            = (known after apply)
    + id             = (known after apply)
    + secret_id      = "[redacted]"
    + secret_string  = (sensitive value)
    + version_id     = (known after apply)
    + version_stages = (known after apply)
  }
Plan: 2 to add, 0 to change, 4 to destroy.
```

Fonte: https://www.bitslovers.com/terraform-plan/

› Terraform Plan –out=plan

- terraform plan previews the changes Terraform will perform before applying them (e.g., create resource A, modify resource B, destroy resource C).
- Validates the configuration and compares the desired state with the current state to detect drift, conflicts, and potential errors.
- The parameter '–out' defines the name of the plan, that will be used next for applying it
- You can also pass variables in the plan command: -var=rg_name=rg1"

Interno

Terraform Apply

```
~/Devops/terraform_test$ terraform apply tfplan
azurerm_resource_group.rg: Creating...
azurerm_resource_group.rg: Still creating... [00m10s elapsed]
azurerm_resource_group.rg: Creation complete after 14s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGrou
azurerm_virtual_network.vnet: Creating...
azurerm_virtual_network.vnet: Creation complete after 6s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGr
rs/Microsoft.Network/virtualNetworks/vnet-demo]
azurerm_subnet.subnet: Creating...
azurerm_subnet.subnet: Creation complete after 9s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGroups/rg
osoft.Network/virtualNetworks/vnet-demo/subnets/subnet-demo]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

resource_group_location = "westeurope"
resource_group_name = "rg-demo"
```

› Terraform apply [plan_name]

- 'apply' generates a plan, ask users' approval and enforces the planned changes
- terraform apply -auto-approve : applies the plan without asking for approval (not recommended for production)

Interno

Terraform Destroy

```
fgpqy6gn2cdn6svngm25tlhp7ya3cdtsj4asq, 20s elapsed]
module.oci-ampere-a1.oci_core_instance.ampere_a1[0]: Still destroying... [id=ocid1.instance.oc1.ca-montreal-1.an4xkl...2
fgpqy6gn2cdn6svngm25tlhp7ya3cdtsj4asq, 30s elapsed]
module.oci-ampere-a1.oci_core_instance.ampere_a1[0]: Still destroying... [id=ocid1.instance.oc1.ca-montreal-1.an4xkl...2
fgpqy6gn2cdn6svngm25tlhp7ya3cdtsj4asq, 40s elapsed]
module.oci-ampere-a1.oci_core_instance.ampere_a1[0]: Still destroying... [id=ocid1.instance.oc1.ca-montreal-1.an4xkl...2
fgpqy6gn2cdn6svngm25tlhp7ya3cdtsj4asq, 50s elapsed]
module.oci-ampere-a1.oci_core_instance.ampere_a1[0]: Still destroying... [id=ocid1.instance.oc1.ca-montreal-1.an4xkl...2
fgpqy6gn2cdn6svngm25tlhp7ya3cdtsj4asq, 1m0s elapsed]
module.oci-ampere-a1.oci_core_instance.ampere_a1[0]: Destruction complete after 1m6s
module.oci-ampere-a1.oci_core_subnet.ampere_subnet: Destroying... [id=ocid1.subnet.oc1.ca-montreal-1.aaaaaaaa2vuexd3xnmv
ozyb4stw7qiccgd2vt2q673zgatem7yays6qttpca]
module.oci-ampere-a1.tls_private_key.oci: Destroying... [id=035426bcfc83c0ac728e5ccd9724799b531bc3f5]
module.oci-ampere-a1.tls_private_key.oci: Destruction complete after 1s
module.oci-ampere-a1.oci_core_subnet.ampere_subnet: Destruction complete after 1s
module.oci-ampere-a1.oci_core_route_table.ampere_route_table: Destroying... [id=ocid1.routetable.oc1.ca-montreal-1.aaaaa
aaa4jf3ilvjzpqlsktbpwrfk7ggl h4e5qtn7soqrdjwx5k7rwztib2q]
module.oci-ampere-a1.oci_core_security_list.ampere_security_list: Destroying... [id=ocid1.securitylist.oc1.ca-montreal-1
.aaaaaaaaaavqczrk64byeuq7jfwciqarkdpeayvftqpmhqt7454eehomn42q]
module.oci-ampere-a1.oci_core_route_table.ampere_route_table: Destruction complete after 1s
module.oci-ampere-a1.oci_core_internet_gateway.ampere_internet_gateway: Destroying... [id=ocid1.internetgateway.oc1.ca-m
ontreal-1.aaaaaaaaf7d3fpxnzbk45lmgpema7zffofghabjtuijxrb3h2ldaqzt4wkaa]
module.oci-ampere-a1.oci_core_security_list.ampere_security_list: Destruction complete after 1s
module.oci-ampere-a1.oci_core_internet_gateway.ampere_internet_gateway: Destruction complete after 0s
module.oci-ampere-a1.oci_core_virtual_network.ampere_vcn: Destroying... [id=ocid1.vcn.oc1.ca-montreal-1.amaaaaaas3ocaxia
x6lm5woif3kpm6vhra2jlrqyeeoqjwovnax5w33322na]
module.oci-ampere-a1.oci_core_virtual_network.ampere_vcn: Destruction complete after 2s

Destroy complete! Resources: 26 destroyed.
```

Fonte: https://amperecomputing.com/blogs/introducing-openmandriva-on-oci-ampere-a1-with-terraform-

> Terraform destroy

- Destroy the infrastructure based on the plan
- List the resources to be destroyed and ask for confirmation
- OBS: terraform destroy is actually a short for "terraform apply -destroy"
- terraform plan –destroy : show the resources to be destroyed without appying it
- terraform destroy -target=[resource_name] : destroy the resource you specify